

Zadanie indywidualne: Obiektość w C# / .NET Core

Klasy, dziedziczenie, interfejs, klasa abstrakcyjna, agregacja i kompozycja

Opis ogólny zadania

Napisz aplikację konsolową w C# w środowisku .NET Core / .NET, która modeluje prosty system obsługi szkoleń. Celem zadania jest przećwiczenie najważniejszych mechanizmów programowania obiektowego w jednym, spójnym przykładzie.

Program ma być wykonany indywidualnie. Nie chodzi o stworzenie dużej aplikacji produkcyjnej, ale o poprawne zaprojektowanie klas, relacji między nimi oraz prostego menu konsolowego.

Scenariusz domenowy

Tworzysz mini-system do obsługi szkolenia „Podstawy .NET”. W systemie mają występować osoby biorące udział w szkoleniu, trenerzy, kursanci, moduły szkoleniowe, harmonogram oraz raport końcowy.

System powinien pozwalać na utworzenie obiektów, wyświetlenie informacji o szkoleniu, dodanie kursantów do szkolenia, dodanie ocen kursantom oraz wyświetlenie prostego podsumowania.

Część 1. Model obiektowy

W pierwszej części należy zaprojektować klasy i relacje między nimi. Ta część może działać na danych wpisanych na sztywno w kodzie, bez rozbudowanego menu.

Wymagane elementy modelu

1. Utwórz klasę abstrakcyjną *Osoba*.
 - Klasa powinna zawierać co najmniej: imię, nazwisko oraz wiek.
 - Klasa powinna posiadać konstruktor ustawiający podstawowe dane osoby.
 - Klasa powinna posiadać zwykłą metodę, np. `WyswietlPodstawoweDane()`.
 - Klasa powinna posiadać metodę abstrakcyjną, np. `WyswietlRole()`, którą klasy potomne będą musiały nadpisać.
2. Utwórz klasę *Kursant* dziedziczącą po klasie *Osoba*.
 - Kursant powinien posiadać dodatkowe pole lub właściwość, np. numer uczestnika.
 - Kursant powinien przechowywać listę ocen.
 - Kursant powinien mieć metodę `DodajOcene(int ocena)`.
 - Kursant powinien mieć metodę `ObliczSrednia()`.
 - Kursant powinien nadpisać metodę abstrakcyjną `WyswietlRole()`.
3. Utwórz klasę *Trener* dziedziczącą po klasie *Osoba*.
 - Trener powinien posiadać dodatkowe pole lub właściwość, np. specjalizacja.
 - Trener powinien nadpisać metodę abstrakcyjną `WyswietlRole()`.
4. Utwórz interfejs *IRaportowalny*.
 - Interfejs powinien zawierać metodę `GenerujRaport()`.
 - Co najmniej jedna klasa powinna implementować ten interfejs, np. *Szkolenie* albo *RaportSzkolenia*.
5. Utwórz klasę *ModulSzkoleniowy*.

- Moduł powinien zawierać nazwę modułu, liczbę godzin oraz krótki opis.
- Przykładowe moduły: „Zmienne i typy danych”, „Klasy i obiekty”, „Dziedziczenie”, „Interfejsy”.

6. Utwórz klasę Harmonogram.

- Harmonogram powinien przechowywać listę modułów szkoleniowych.
- Powinien posiadać metodę DodajModul(ModulSzkoleniowy modul).
- Powinien posiadać metodę WyświetlHarmonogram().

7. Utwórz klasę Szkolenie.

- Szkolenie powinno posiadać nazwę szkolenia.
- Szkolenie powinno posiadać trenera.
- Szkolenie powinno posiadać listę kursantów.
- Szkolenie powinno posiadać harmonogram.
- Szkolenie powinno implementować interfejs IRaportowalny albo korzystać z osobnej klasy raportu implementującej ten interfejs.

Relacje obiektowe, które muszą wystąpić

Pojęcie	Jak ma wystąpić w zadaniu	Przykład interpretacji
Klasa	Utwórz własne typy danych.	Osoba, Kursant, Trener, Szkolenie, Harmonogram.
Dziedziczenie	Klasy szczegółowe mają dziedziczyć po klasie ogólnej.	Kursant i Trener dziedziczą po Osoba.
Klasa abstrakcyjna	Nie wolno tworzyć obiektu klasy Osoba bezpośrednio.	Osoba jest tylko wspólną bazą dla konkretnych typów osób.
Interfejs	Wybrana klasa ma podpisać kontrakt.	Szkolenie implementuje IRaportowalny i posiada metodę GenerujRaport().
Agregacja	Jeden obiekt ma referencje do obiektów istniejących niezależnie.	Szkolenie ma listę kursantów. Kursanci mogą istnieć także poza szkoleniem.
Kompozycja	Jeden obiekt tworzy i posiada swoją część wewnętrzną.	Szkolenie tworzy własny Harmonogram w konstruktorze.

Część 2. Program konsolowy

W drugiej części należy użyć zaprojektowanych klas w aplikacji konsolowej. Program powinien działać w sposób czytelny dla użytkownika.

Minimalne funkcje programu

1. Po uruchomieniu program powinien utworzyć przykładowe szkolenie, np. „Podstawy .NET”.
2. Program powinien utworzyć przykładowego trenera.
3. Program powinien pozwolić dodać kilku kursantów do szkolenia.
4. Program powinien pozwolić dodać oceny wybranemu kursantowi.
5. Program powinien wyświetlić listę wszystkich kursantów.
6. Program powinien wyświetlić harmonogram szkolenia.

7. Program powinien wygenerować raport końcowy szkolenia przy pomocy metody z interfejsu IRaportowalny.
8. Program powinien pokazać średnią ocen każdego kursanta.
9. Program powinien wskazać kursanta z najwyższą średnią.

Proponowane menu programu

1. Wyświetl informacje o szkoleniu
2. Wyświetl trenera
3. Dodaj kursanta
4. Wyświetl kursantów
5. Dodaj ocenę kursantowi
6. Wyświetl harmonogram
7. Wygeneruj raport szkolenia
8. Zakończ program

Wymagania techniczne

- Projekt powinien być aplikacją konsolową w C# utworzoną jako Console App w .NET Core / .NET.
- Każda większa klasa powinna znajdować się w osobnym pliku .cs.
- Nazwy klas, metod i zmiennych powinny być czytelne.
- Program powinien wykorzystywać List<T> do przechowywania kursantów, ocen oraz modułów szkoleniowych.
- Program powinien zawierać proste zabezpieczenie przed błędnymi danymi, np. sprawdzenie czy ocena mieści się w zakresie 1-6.
- Nie należy pisać całego programu w metodzie Main. Metoda Main powinna tylko uruchamiać logikę programu i obsługiwać menu.

Sugerowana struktura plików

- Program.cs
- Osoba.cs
- Kursant.cs
- Trener.cs
- IRaportowalny.cs
- ModulSzkoleniowy.cs
- Harmonogram.cs
- Szkolenie.cs

Warunki zaliczenia zadania

1. Program kompiluje się i uruchamia bez błędów.
2. W kodzie występuje co najmniej jedna klasa abstrakcyjna.
3. W kodzie występuje dziedziczenie po klasie abstrakcyjnej.
4. W kodzie występuje co najmniej jeden interfejs i jego implementacja.
5. W kodzie występuje agregacja, np. Szkolenie zawiera listę kursantów.

6. W kodzie występuje kompozycja, np. Szkolenie tworzy własny Harmonogram.
7. Program wykorzystuje kolekcje List<T>.
8. Program posiada czytelne menu konsolowe albo przynajmniej czytelny scenariusz testowy w Main.
9. Kod jest podzielony na klasy i pliki, a nie napisany jako jeden długi blok instrukcji.

Wersja rozszerzona dla chętnych

- Dodaj możliwość usuwania kursanta ze szkolenia.
- Dodaj możliwość wyszukiwania kursanta po nazwisku.
- Dodaj wyliczenie enum PoziomZaawansowania z wartościami: Podstawowy, Sredni, Zaawansowany.
- Dodaj klasę Certyfikat tworzoną dla kursanta po ukończeniu szkolenia.
- Dodaj zapis raportu szkolenia do pliku tekstowego.
- Dodaj obsługę wyjątków lub TryParse przy wczytywaniu danych liczbowych.

Uwaga końcowa

Najważniejsze w tym zadaniu nie jest samo menu, ale poprawne pokazanie relacji obiektowych. Program ma udowodnić, że rozumiesz różnicę między dziedziczeniem, interfejsem, klasą abstrakcyjną, agregacją i kompozycją.